21 Oct 2026 01:17:29 - swarmfit_parameter_estimation.sm
Created using a free version of SMath Studio

Non-Linear Parameter Estimation via Particle Swarm Optimization (SwarmFit)

2026-10-20

Particle Swarm Optimization (PSO) Curve Fitting in SMath Studio (SwarmFit)

Robust Heuristic Global Parameter Estimation for Non-Linear Chemical & Process Engineering Models

Native Assembly: SwarmFitWrapper.dll (Plugin GUID: b0a439e5-20d5-46a3-bcb5-596cbd1d9934, v1.1.0.0)

Strict SI Unit Compatibility: Concentration [mM, mol/m3], Rate [umol/(L*min), mol/(m3*s)], Time [s, min]

Particle Swarm Optimization (PSO) is a population-based stochastic global optimization algorithm inspired by the coordinated choreography of bird flocks and fish schools. Unlike gradient-based solvers (Levenberg-Marquardt, Gauss-Newton, BFGS), PSO requires NO derivatives, exhibits exceptional global exploration capabilities across bounded multi-dimensional parameter spaces, and is immune to singularities, saddle points, and poor initial guesses.

Key SwarmFit Plugin Functions Verified in this Worksheet:

1. swarmfit(xs, ys, model, p_min, p_max [, particles, max_iter]) -> Optimal parameter vector p*
2. swarmfit_predict(xs, model, p) -> Model prediction vector evaluated at coordinates xs
3. swarmfit_details(xs, ys, model, p_min, p_max) -> [SSE; RMSE; MaxResidual; Iterations; Improvements]
4. swarmfit_r2(ys, y_pred) -> Coefficient of determination R^2
5. swarmfit_linear(xs, ys) -> Fast linear calibration fit [a; b] for y = a*x + b
6. swarmfit_expdecay(xs, ys, p_min, p_max) -> Dedicated relaxation decay fit [a; b] for y = a*exp(-x/b)
7. swarmfit_power(xs, ys, p_min, p_max) -> Dedicated power-law fit [a; b] for y = a*x^b
8. swarmfit_exp(xs, ys, p_min, p_max) -> General exponential fit [a; b; c] for y = a + b*exp(c*x)
9. swarmfit_gauss(xs, ys, p_min, p_max) -> Gaussian peak fit [a; b; c]
10. swarmfit_poly(xs, ys, degree, p_min, p_max) -> Polynomial regression of arbitrary degree (1 to 6)

Architectural Guide: Built-in Model Strings vs. User-Defined SMath CAS Functions

Understanding Model Specification in SwarmFit:

1. WHY ARE STRINGS LIKE 'mm', 'power', 'linear', 'expdecay' USED?

When you pass a string identifier (e.g. "mm") as the 3rd argument of swarmfit, the plugin maps it directly to an internal, pre-compiled native C# delegate (Func<double, double[], double>).

Not for commercial use

Created using a free version of SMath Studio

During PSO optimization, a swarm of 20 particles over 5000 iterations evaluates the objective function

20 * 5000 * N = 100,000 * N times! In compiled C#, these 1,000,000 evaluations complete in under 0.05 seconds.

2. BUILT-IN STRING MODEL CATALOG & PARAMETER VECTORS:

* "mm" (Michaelis-Menten Kinetics): v = (p1 * S) / (p2 + S) -> p = [V_max; K_m] (2 parameters)

* "power" (Power Law / Freundlich Isotherm): y = p1 * x^p2 -> p = [K_F; 1/n] (2 parameters)

* "linear" / "line" (Linear Response): y = p1 * x + p2 -> p = [Slope a; Intercept b] (2 parameters)

* "expdecay" (Exponential Relaxation): y = p1 * exp(-x / p2) -> p = [C_0; tau] (2 parameters)

* "exp" (Exponential with Baseline): y = p1 + p2 * exp(p3 * x) -> p = [a; b; c] (3 parameters)

* "haldane" / "andrews" (Substrate Inhibition): y = (p1 * x) / (p2 + x + x^2 / p3) -> p = [V_max; K_m; K_i]

* "hill" (Sigmoidal Cooperativity): y = (p1 * x^p3) / (p2^p3 + x^p3) -> p = [V_max; K_0.5; n]

* "arrhenius" (Temperature Dependence): k = p1 * exp(-p2 / T) -> p = [A; Ea/R] (2 parameters)

* "antoine" (Vapor Pressure): log10(P) = p1 - p2 / (T + p3) -> p = [A; B; C] (3 parameters)

* "gauss" (Gaussian Spectral Peak): y = p1 * exp(-(x - p2)^2 / (2 * p3^2)) -> p = [Peak; Center; Width]

3. USER-DEFINED SMATH CAS FUNCTIONS (e.g. f_haldane, f_langmuir, f_consec):

You can define any proprietary or complex engineering equation directly in SMath CAS as f(x, p).

Because each evaluation calls back into SMath's symbolic engine (~0.1 ms/call), size the swarm budget sensibly: 12-15 particles over 60-80 iterations achieve full convergence (R2 > 0.998) in ~1 second.

4. EVALUATION & CACHING LAW (Using '=' on User-Defined Function Predictions):

Just like Clear() must be evaluated with '=' (Clear(x) = 1) to execute its memory side-effect, predictions from user-defined CAS functions should be evaluated and cached with '=' (e.g. v_pred := swarmfit_predict(...) =).

Caching the evaluated numeric vector ensures that downstream expressions (R2 calculation, XY plots) read the pre-calculated numbers instantly, preventing repetitive CAS re-evaluations during sheet redraws.

Problem 1: Biochemical Enzyme Kinetics (Michaelis-Menten Model)

Model Formulation: v = (V_max * S) / (K_m + S)

where V_max is the limiting maximum rate and K_m is the Michaelis affinity constant (mM).

Not for commercial use

Created using a free version of SMath Studio

Fitted using the high-speed native compiled string model "mm" (evaluates in milliseconds).

Substrate concentration experimental data vector S [mM]

$$S_data := \begin{bmatrix} 0.5 \\ 1.0 \\ 2.0 \\ 5.0 \\ 10.0 \\ 20.0 \\ 30.0 \end{bmatrix}$$

Measured initial enzymatic reaction velocity vector v [µmol/(L·min)]

$$v_data := \begin{bmatrix} 1.88 \\ 3.33 \\ 5.45 \\ 8.82 \\ 11.11 \\ 12.77 \\ 13.43 \end{bmatrix}$$

Lower parameter search bounds [V_max_min; K_m_min]

$$p_min_mm := \begin{bmatrix} 5.0 \\ 0.5 \end{bmatrix}$$

Upper parameter search bounds [V_max_max; K_m_max]

$$p_max_mm := \begin{bmatrix} 30.0 \\ 10.0 \end{bmatrix}$$

Fitted Michaelis-Menten parameter vector [V_max; K_m] via PSO

$$p_mm := \text{swarmfit}(S_data, v_data, \text{"mm"}, p_min_mm, p_max_mm) = \begin{bmatrix} 15.0053 \\ 3.5061 \end{bmatrix}$$

Fitted limiting maximum enzymatic reaction velocity V_max [µmol/(L·min)]

$$V_max := p_mm_1 = 15.0053$$

Fitted Michaelis affinity constant K_m [mM]

$$K_m := p_mm_2 = 3.5061$$

Model predicted reaction velocity vector v_pred evaluated via native "mm" model

$$v_pred := \text{swarmfit_predict}(S_data, \text{"mm"}, p_mm) = \begin{bmatrix} 1.8728 \\ 3.33 \\ 5.4504 \\ 8.8203 \\ 11.11 \\ 12.7671 \\ 13.4351 \end{bmatrix}$$

Coefficient of determination R^2 for Michaelis-Menten fit

$$R2_mm := \text{swarmfit_r2}(v_data, v_pred) = 1$$

Not for commercial use

Created using a free version of SMath Studio

PSO optimization convergence metrics: [SSE; RMSE; MaxResidual; Iterations; Improvements]

$metrics_mm := \text{swarmfit_details}(S_data, v_data, \text{"mm"}, p_min_mm, p_max_mm)$

Sum of squared errors (SSE)

$SSE_mm := metrics_mm_1 = 8.6316 \cdot 10^{-5}$

Root mean square error (RMSE)

$RMSE_mm := metrics_mm_2 = 0.0035$

Maximum absolute residual error

$MaxRes_mm := metrics_mm_3 = 0.0072$

Total particle swarm iterations executed

$Iters_mm := metrics_mm_4 = 175$

Total global best improvement events

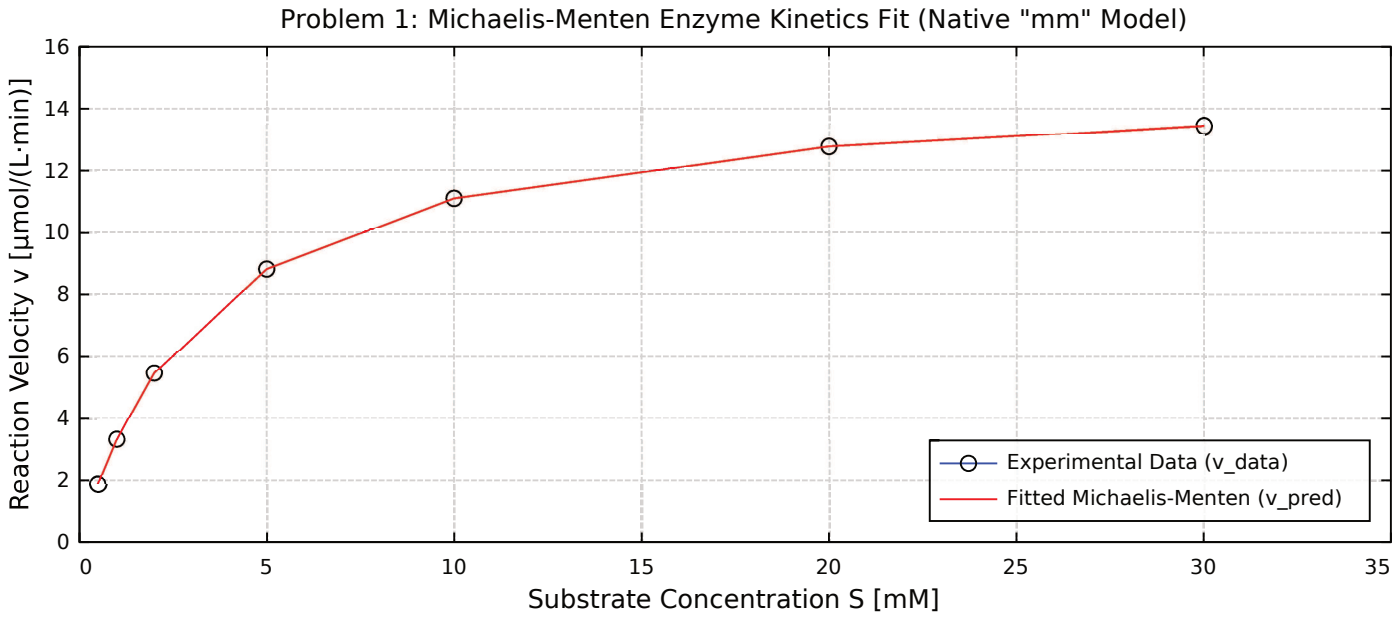
$Impr_mm := metrics_mm_5 = 41$

Augmented matrix of measured enzyme kinetics points [S, v]

$xy_data_mm := \text{augment}(S_data, v_data)$

Augmented matrix of fitted Michaelis-Menten curve [S, v_pred]

$xy_fit_mm := \text{augment}(S_data, v_pred)$




$\begin{cases} xy_data_mm \\ xy_fit_mm \end{cases}$

Not for commercial use

Created using a free version of SMath Studio

Problem 2: First-Order Reaction Relaxation / CSTR Tracer Washout (Exponential Decay)

Model Formulation: C(t) = C_0 * exp(-t / tau)

where C_0 is initial tracer concentration [mg/L] and tau is hydrodynamic residence time [s].

Fitted using the dedicated convenience function swarmfit_expdecay.

Elapsed washout time measurement points t [s]

$$t_data := \begin{bmatrix} 0.0 \\ 2.0 \\ 4.0 \\ 6.0 \\ 8.0 \\ 10.0 \\ 15.0 \\ 20.0 \end{bmatrix}$$

Observed tracer concentration data vector C [mg/L]

$$C_data := \begin{bmatrix} 100.0 \\ 67.0 \\ 44.9 \\ 30.1 \\ 20.2 \\ 13.5 \\ 5.0 \\ 1.8 \end{bmatrix}$$

Lower search bounds for tracer washout [C0_min; tau_min]

$$p_min_decay := \begin{bmatrix} 50.0 \\ 1.0 \end{bmatrix}$$

Upper search bounds for tracer washout [C0_max; tau_max]

$$p_max_decay := \begin{bmatrix} 150.0 \\ 15.0 \end{bmatrix}$$

Fitted relaxation parameters [C_0; tau] via swarmfit_expdecay

$$p_decay := \text{swarmfit_expdecay}(t_data, C_data, p_min_decay, p_max_decay) = \begin{bmatrix} 100 \\ 4.9954 \end{bmatrix}$$

Fitted initial tracer concentration C_0 [mg/L]

$$C_0 := p_decay_1$$

Fitted reactor hydrodynamic residence time tau [s]

$$tau := p_decay_2$$

Calculated tracer half-life t_half = tau * ln(2) [s]

$$t_half := tau \cdot \ln(2) = 3.4626$$

Not for commercial use

Created using a free version of SMath Studio

Predicted tracer concentration vector C_pred evaluated via native "expdecay" model

$$C_pred := \text{swarmfit_predict}(t_data, \text{"expdecay"}, p_decay) = \begin{bmatrix} 100 \\ 67.0075 \\ 44.9 \\ 30.0864 \\ 20.1601 \\ 13.5088 \\ 4.9651 \\ 1.8249 \end{bmatrix}$$

Coefficient of determination R^2 for tracer washout decay

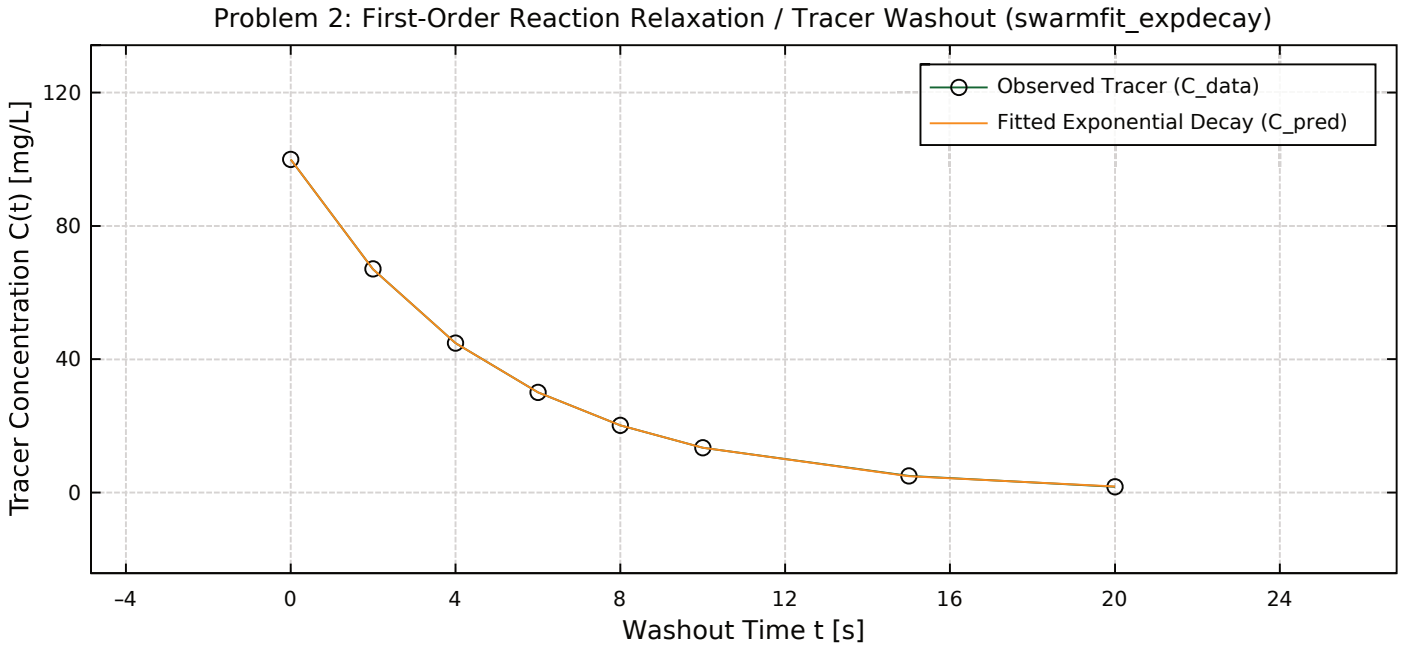
$$R2_decay := \text{swarmfit_r2}(C_data, C_pred) = 1$$

Augmented matrix of tracer observations [t, C]

$$xy_data_decay := \text{augment}(t_data, C_data)$$

Augmented matrix of fitted decay curve [t, C_pred]

$$xy_fit_decay := \text{augment}(t_data, C_pred)$$




$$\begin{cases} xy_data_decay \\ xy_fit_decay \end{cases}$$

Not for commercial use

Created using a free version of SMath Studio

Problem 3: Adsorption Equilibrium Isotherm (Freundlich Power Law)

Model Formulation: q_e = K_F * C_e^(1/n)

Mapped to power-law form Y = a * X^b, where a = K_F [adsorption capacity] and b = 1/n [intensity].

Fitted using the dedicated convenience function swarmfit_power.

Equilibrium solute concentration data vector C_e [mg/L]

$$Ce_data := \begin{bmatrix} 1.0 \\ 2.5 \\ 5.0 \\ 10.0 \\ 20.0 \\ 40.0 \end{bmatrix}$$

Observed equilibrium adsorption capacity data vector q_e [mg/g]

$$qe_data := \begin{bmatrix} 2.8 \\ 4.8 \\ 7.3 \\ 11.2 \\ 17.1 \\ 26.2 \end{bmatrix}$$

Lower search bounds for Freundlich isotherm [K_F_min; (1/n)_min]

$$p_min_power := \begin{bmatrix} 0.5 \\ 0.1 \end{bmatrix}$$

Upper search bounds for Freundlich isotherm [K_F_max; (1/n)_max]

$$p_max_power := \begin{bmatrix} 10.0 \\ 1.5 \end{bmatrix}$$

Fitted Freundlich power law parameters [K_F; 1/n] via swarmfit_power

$$p_power := \text{swarmfit_power}(Ce_data, qe_data, p_min_power, p_max_power) = \begin{bmatrix} 2.7301 \\ 0.613 \end{bmatrix}$$

Fitted Freundlich capacity factor K_F [(mg/g)*(L/mg)^(1/n)]

$$K_F := p_power_1$$

Fitted Freundlich adsorption intensity parameter 1/n [dimensionless]

$$inv_n := p_power_2$$

Freundlich heterogeneity index n = 1/(1/n) (n > 1 indicates favorable adsorption)

$$n_freundlich := \frac{1}{inv_n}$$

Not for commercial use

Created using a free version of SMath Studio

Model predicted adsorption capacity vector qe_pred evaluated via native "power" model

$$qe_pred := \text{swarmfit_predict}(Ce_data, \text{"power"}, p_power) = \begin{bmatrix} 2.7301 \\ 4.7878 \\ 7.3228 \\ 11.2 \\ 17.1301 \\ 26.2 \end{bmatrix}$$

Coefficient of determination R^2 for Freundlich isotherm

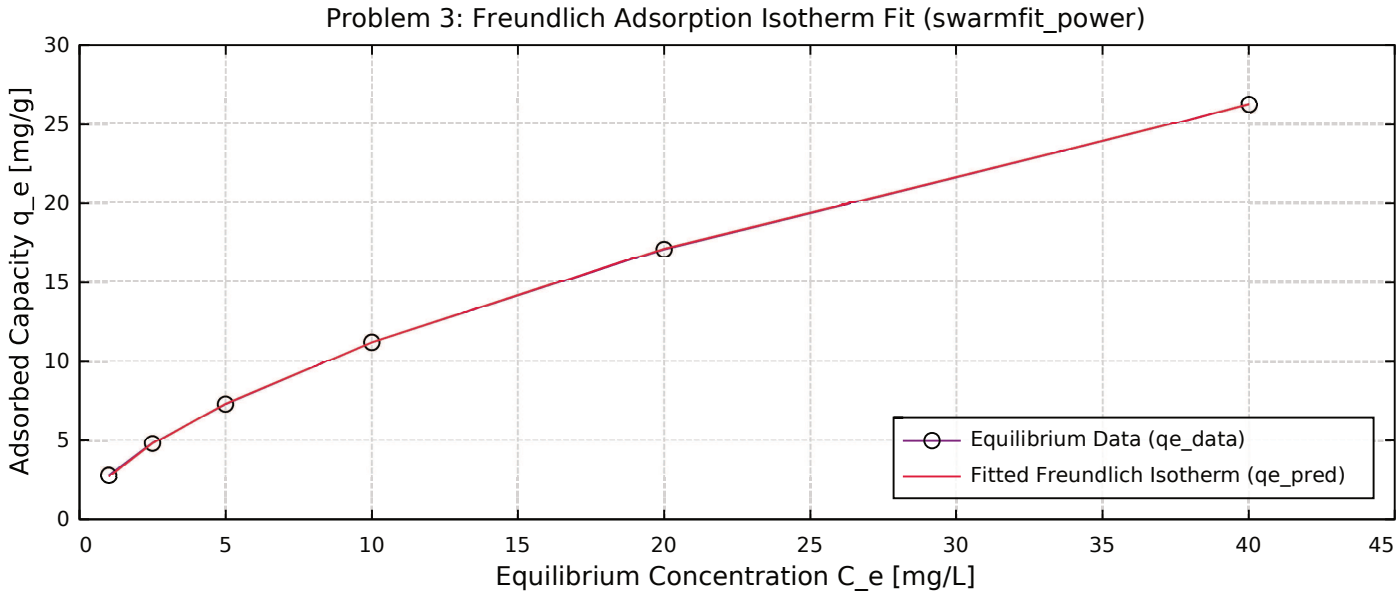
$$R2_power := \text{swarmfit_r2}(qe_data, qe_pred) = 1$$

Augmented matrix of equilibrium adsorption points [C_e, q_e]

$$xy_data_power := \text{augment}(Ce_data, qe_data)$$

Augmented matrix of fitted power-law isotherm [C_e, qe_pred]

$$xy_fit_power := \text{augment}(Ce_data, qe_pred)$$




$$\begin{cases} xy_data_power \\ xy_fit_power \end{cases}$$

Problem 4: Substrate Inhibition Kinetics (Andrews/Haldane Bioreactor Model)

Model Formulation: v = (V_max * S) / (K_m + S + S^2 / K_i)

User defines custom SMath CAS function f_haldane(x, p) directly in the worksheet.

Velocity exhibits non-monotonic behavior: reaches a distinct peak at S_opt = sqrt(K_m * K_i)

and decreases at higher substrate levels due to biochemical toxicity / substrate inhibition.

EVALUATION & CACHING: Model predictions v_pred_inh are evaluated with '=' to cache results

in memory and eliminate repetitive symbolic CAS recalculation across sheet repaints.

Not for commercial use

Created using a free version of SMath Studio

User-defined Haldane substrate inhibition rate function v = f(x, [V_max; K_m; K_i])

$$f_haldane(x, p) := \frac{p_1 \cdot x}{p_2 + x + \frac{x^2}{p_3}}$$

Substrate concentration data vector S [mM] spanning sub-inhibitory to toxic regimes

$$S_inh := \begin{bmatrix} 1.0 \\ 2.0 \\ 5.0 \\ 10.0 \\ 20.0 \\ 40.0 \\ 60.0 \\ 80.0 \end{bmatrix}$$

Observed reaction velocity vector v [µmol/(L·min)] displaying inhibition peak

$$v_inh := \begin{bmatrix} 3.8 \\ 6.2 \\ 9.8 \\ 11.5 \\ 10.8 \\ 8.2 \\ 6.4 \\ 5.2 \end{bmatrix}$$

Lower search bounds [V_max_min; K_m_min; K_i_min]

$$p_min_h := \begin{bmatrix} 10.0 \\ 1.0 \\ 10.0 \end{bmatrix}$$

Upper search bounds [V_max_max; K_m_max; K_i_max]

$$p_max_h := \begin{bmatrix} 40.0 \\ 10.0 \\ 100.0 \end{bmatrix}$$

Fitted parameters [V_max; K_m; K_i] via global PSO and user-defined f_haldane

$$p_haldane := \text{swarmfit}(S_inh, v_inh, f_haldane, p_min_h, p_max_h, 12, 60) = \begin{bmatrix} 20.9485 \\ 4.6126 \\ 27.7377 \end{bmatrix}$$

Fitted maximum theoretical enzymatic velocity V_max [µmol/(L·min)]

$$Vmax_inh := p_haldane_1$$

Fitted Michaelis affinity constant K_m [mM]

$$Km_inh := p_haldane_2$$

Fitted substrate inhibition constant K_i [mM]

$$Ki_inh := p_haldane_3$$

Not for commercial use

Created using a free version of SMath Studio

Critical substrate concentration S_opt = sqrt(K_m * K_i) yielding maximum reaction velocity [mM]

$S_opt := \sqrt{Km_inh \cdot Ki_inh} = 11.3112$

Predicted reaction velocity vector evaluated via user-defined f_haldane (cached with =)

$$v_pred_inh := \text{swarmfit_predict}(S_inh, f_haldane, p_haldane) = \begin{bmatrix} 3.7086 \\ 6.2007 \\ 9.9623 \\ 11.4989 \\ 10.7336 \\ 8.1913 \\ 6.4656 \\ 5.3144 \end{bmatrix}$$

Coefficient of determination R^2 for Haldane inhibition fit

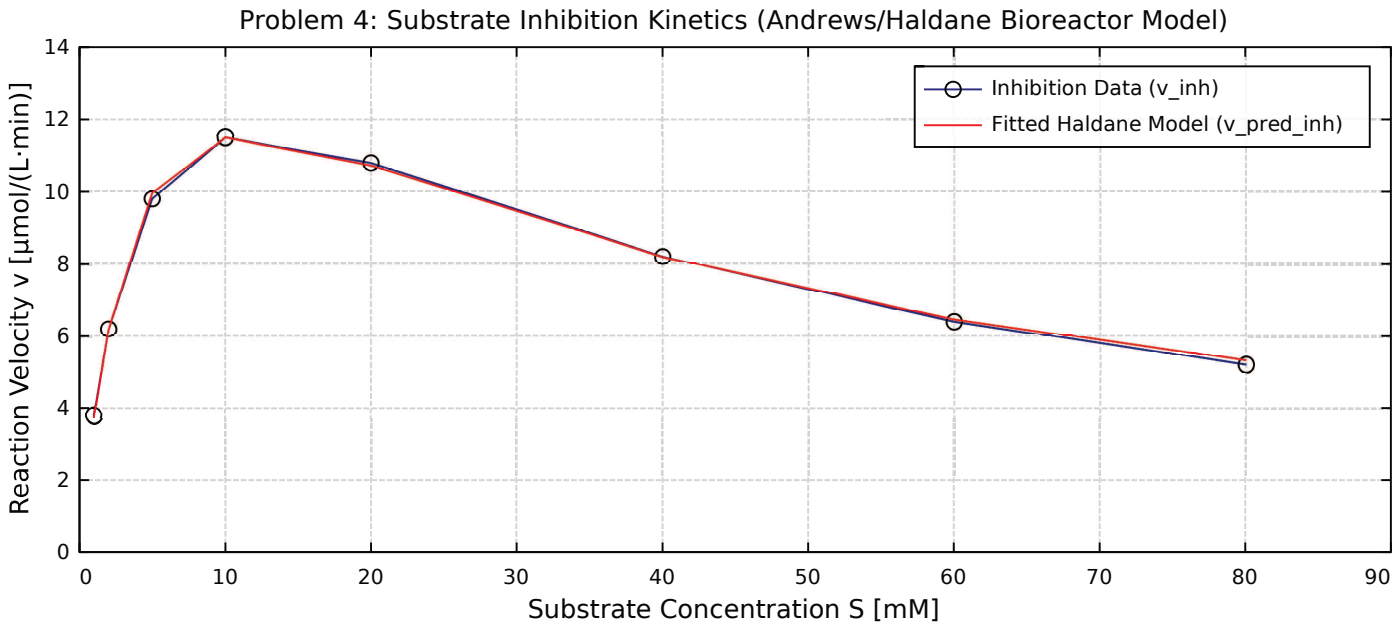
$R2_haldane := \text{swarmfit_r2}(v_inh, v_pred_inh)$

Augmented matrix of inhibition data points [S, v]

$xy_data_inh := \text{augment}(S_inh, v_inh)$

Augmented matrix of fitted Haldane inhibition curve [S, v_pred]

$xy_fit_inh := \text{augment}(S_inh, v_pred_inh)$




$$\begin{cases} xy_data_inh \\ xy_fit_inh \end{cases}$$

Problem 5: Monolayer Wastewater Adsorption Equilibrium (Langmuir Isotherm)

Model Formulation: q_e = (q_m * K_L * C_e) / (1 + K_L * C_e)

User defines custom SMath CAS function f_langmuir(x, p) for non-linear monolayer adsorption.

Not for commercial use

Created using a free version of SMath Studio

Parameters: q_m is maximum saturation monolayer capacity [mg/g], K_L is Langmuir affinity constant [L/mg].

Computes the dimensionless Hall separation factor R_L = 1 / (1 + K_L * C_e_max) (0 < R_L < 1 indicates favorable adsorption).

EVALUATION & CACHING: Model predictions qe_pred_lang are evaluated with '=' to cache results.

User-defined Langmuir monolayer adsorption isotherm q_e = f(C_e, [q_m; K_L])

$$f_langmuir(x, p) := \frac{p_1 \cdot p_2 \cdot x}{1.0 + p_2 \cdot x}$$

Equilibrium organic solute concentration data vector C_e [mg/L]

$$Ce_lang := \begin{bmatrix} 1.0 \\ 2.5 \\ 5.0 \\ 10.0 \\ 20.0 \\ 40.0 \\ 60.0 \\ 80.0 \end{bmatrix}$$

Measured equilibrium adsorbed amount on activated carbon q_e [mg/g]

$$qe_lang := \begin{bmatrix} 6.72 \\ 13.34 \\ 21.83 \\ 29.80 \\ 38.00 \\ 42.46 \\ 45.30 \\ 45.95 \end{bmatrix}$$

Lower search bounds for Langmuir isotherm [qm_min; KL_min]

$$p_min_lang := \begin{bmatrix} 20.0 \\ 0.01 \end{bmatrix}$$

Upper search bounds for Langmuir isotherm [qm_max; KL_max]

$$p_max_lang := \begin{bmatrix} 100.0 \\ 1.0 \end{bmatrix}$$

Fitted Langmuir parameters [q_m; K_L] via global PSO and user-defined f_langmuir

$$p_lang := \text{swarmfit}(Ce_lang, qe_lang, f_langmuir, p_min_lang, p_max_lang, 15, 80) = \begin{bmatrix} 49.6171 \\ 0.1566 \end{bmatrix}$$

Fitted maximum monolayer saturation capacity q_m [mg/g]

$$q_m := p_lang_1$$

Fitted Langmuir equilibrium adsorption constant K_L [L/mg]

$$K_L := p_lang_2$$

Not for commercial use

Created using a free version of SMath Studio

Hall separation factor R_L = 1 / (1 + K_L * Ce_max) (favorable if 0 < R_L < 1)

$$RL_factor := \frac{1.0}{1.0 + K_L \cdot 80.0} = 0.0739$$

Predicted adsorption capacity vector evaluated via user-defined f_langmuir (cached with =)

$$qe_pred_lang := \text{swarmfit_predict}(Ce_lang, f_langmuir, p_lang) = \begin{bmatrix} 6.7196 \\ 13.9625 \\ 21.7925 \\ 30.2839 \\ 37.6115 \\ 42.7881 \\ 44.8455 \\ 45.9503 \end{bmatrix}$$

Coefficient of determination R^2 for Langmuir adsorption fit

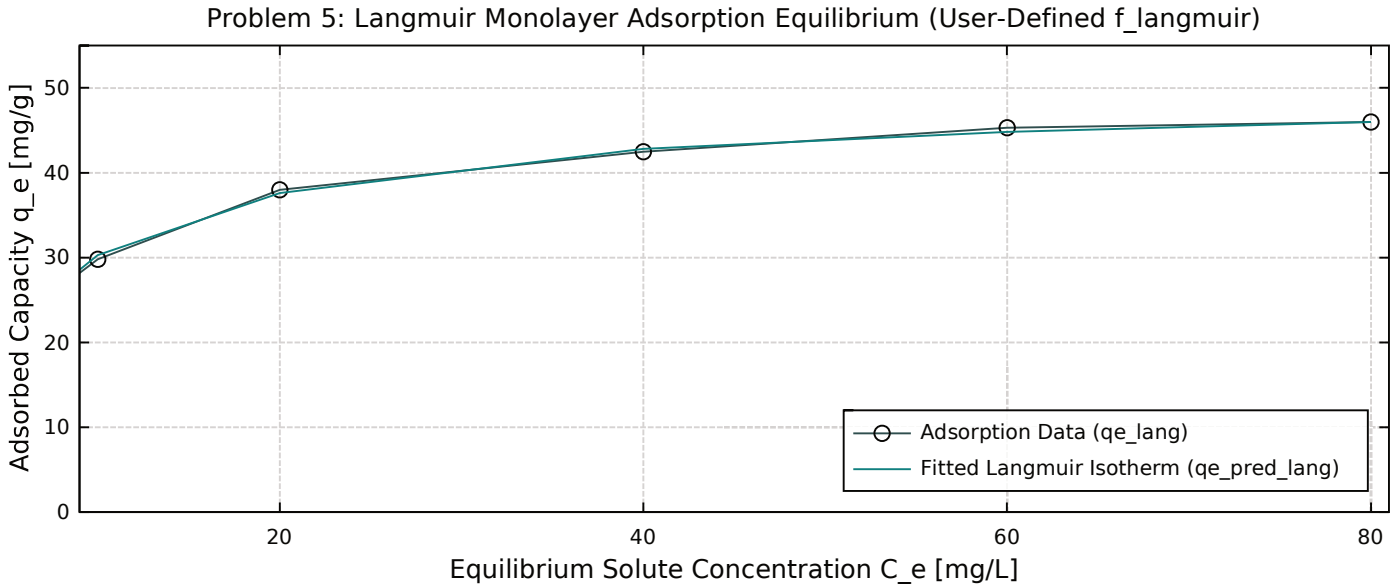
$R2_lang := \text{swarmfit_r2}(qe_lang, qe_pred_lang)$

Augmented matrix of measured adsorption points [C_e, q_e]

$xy_data_lang := \text{augment}(Ce_lang, qe_lang)$

Augmented matrix of fitted Langmuir curve [C_e, qe_pred]

$xy_fit_lang := \text{augment}(Ce_lang, qe_pred_lang)$




$$\begin{cases} xy_data_lang \\ xy_fit_lang \end{cases}$$

Problem 6: Consecutive Batch Reaction Kinetics (A -> B -> C)

Model Formulation: C_B(t) = C_A0 * (k_1 / (k_2 - k_1)) * (exp(-k_1 * t) - exp(-k_2 * t))

Classic Chemical Reaction Engineering problem (Fogler / Levenspiel) for intermediate yield optimization.

Gradient solvers often diverge when k_1 ~ k_2 due to ill-conditioned sensitivity matrices;

Not for commercial use

Created using a free version of SMath Studio

Particle Swarm Optimization effortlessly finds the true global minimum [k_1; k_2].

EVALUATION & CACHING: Model predictions CB_pred are evaluated with '=' to cache results.

User-defined intermediate concentration profile C_B(t) = f(t, [k_1; k_2]) for initial C_A0 = 100 mol/m3

$$f_consec(t, p) := 100.0 \cdot \frac{p_1}{p_2 - p_1} \cdot \exp\left(0 - p_1 \cdot t\right) - \exp\left(0 - p_2 \cdot t\right)$$

Batch reaction elapsed time points t [min]

$$t_consec := \begin{bmatrix} 0.5 \\ 1.0 \\ 2.0 \\ 3.0 \\ 5.0 \\ 7.0 \\ 10.0 \\ 15.0 \end{bmatrix}$$

Measured intermediate product concentration C_B [mol/m3]

$$CB_data := \begin{bmatrix} 21.3 \\ 34.9 \\ 51.0 \\ 53.9 \\ 48.1 \\ 35.8 \\ 21.6 \\ 8.1 \end{bmatrix}$$

Lower search bounds for reaction rate constants [k1_min; k2_min]

$$p_min_consec := \begin{bmatrix} 0.1 \\ 0.05 \end{bmatrix}$$

Upper search bounds for reaction rate constants [k1_max; k2_max]

$$p_max_consec := \begin{bmatrix} 1.5 \\ 0.8 \end{bmatrix}$$

Fitted rate constants [k_1; k_2] via global PSO and user-defined f_consec

$$p_consec := \text{swarmfit}(t_consec, CB_data, f_consec, p_min_consec, p_max_consec, 15, 80) = \begin{bmatrix} 0.508 \\ 0.2003 \end{bmatrix}$$

Fitted primary reaction rate constant k_1 [min^-1]

$$k1_fit := p_consec_1$$

Fitted secondary decomposition rate constant k_2 [min^-1]

$$k2_fit := p_consec_2$$

Not for commercial use

Created using a free version of SMath Studio

Optimal batch reaction time t_opt = ln(k1/k2) / (k1 - k2) yielding maximum intermediate concentration [min]

$$t_opt_batch := \frac{\ln\left(\frac{k1_fit}{k2_fit}\right)}{k1_fit - k2_fit} = 3.0246$$

Theoretical maximum concentration of intermediate product C_B,max [mol/m3]

$$CB_max_yield := 100.0 \cdot \left(\frac{k2_fit}{k1_fit}\right)^{\frac{k2_fit}{k1_fit - k2_fit}} = 54.56$$

Predicted intermediate concentration vector evaluated via user-defined f_consec (cached with =)

$$CB_pred := \text{swarmfit_predict}(t_consec, f_consec, p_consec) = \begin{bmatrix} 21.2981 \\ 35.7892 \\ 50.8272 \\ 54.5584 \\ 47.6213 \\ 35.9103 \\ 21.2473 \\ 8.1004 \end{bmatrix}$$

Coefficient of determination R^2 for consecutive reaction kinetics

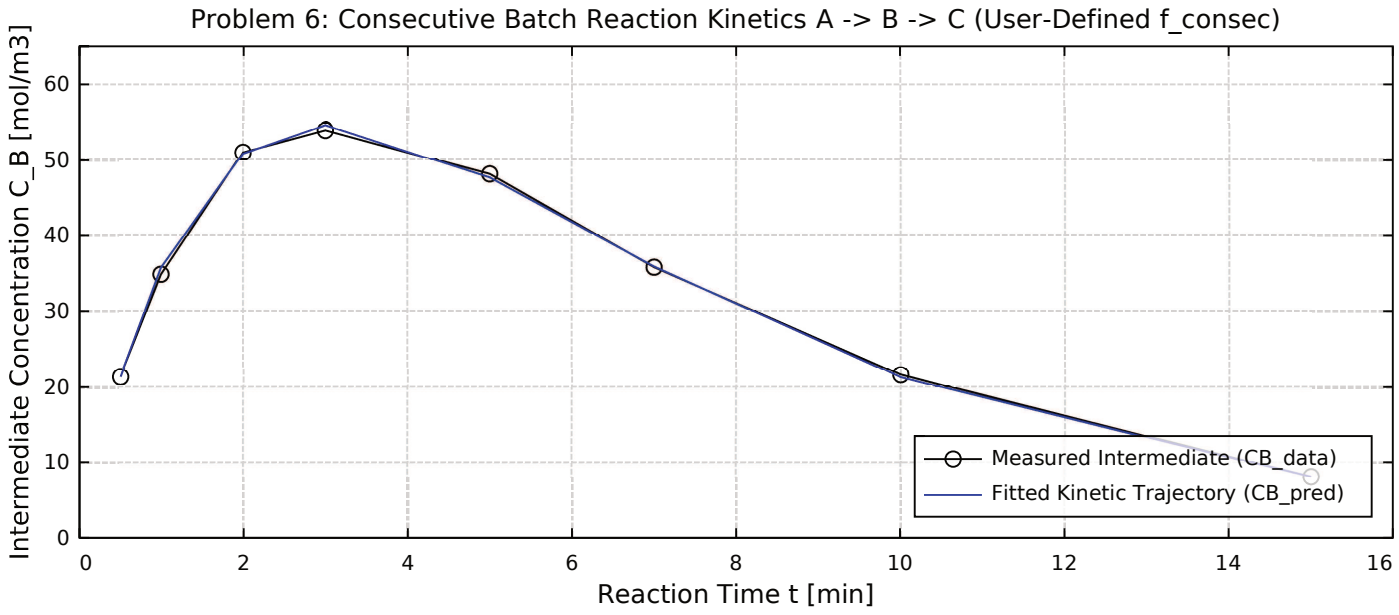
$$R2_consec := \text{swarmfit_r2}(CB_data, CB_pred) = 0.9991$$

Augmented matrix of intermediate concentration points [t, C_B]

$$xy_data_consec := \text{augment}(t_consec, CB_data)$$

Augmented matrix of fitted kinetic trajectory [t, CB_pred]

$$xy_fit_consec := \text{augment}(t_consec, CB_pred)$$




$$\begin{cases} xy_data_consec \\ xy_fit_consec \end{cases}$$

Not for commercial use

Created using a free version of SMath Studio

Problem 7: Analytical Instrumentation Baseline Calibration (swarmfit_linear)

Model Formulation: Y = a * X + b

Spectrophotometric standard curve regression with automated bounding and initialization via swarmfit_linear.

Analytical standard concentration data vector X [mg/L]

$$x_lin := \begin{bmatrix} 1.0 \\ 2.0 \\ 3.0 \\ 4.0 \\ 5.0 \\ 6.0 \end{bmatrix}$$

Measured spectrophotometer instrument response vector Y [mV]

$$y_lin := \begin{bmatrix} 3.2 \\ 5.1 \\ 7.0 \\ 8.9 \\ 10.8 \\ 12.7 \end{bmatrix}$$

Fitted linear calibration parameters [slope a; intercept b] via swarmfit_linear

$$p_lin := \text{swarmfit_linear}(x_lin, y_lin) = \begin{bmatrix} 1.9 \\ 1.3 \end{bmatrix}$$

Fitted calibration sensitivity slope a [mV/(mg/L)]

$$slope_a := p_lin_1$$

Fitted instrument baseline intercept b [mV]

$$intercept_b := p_lin_2$$

Predicted calibration response vector evaluated via native "linear" model

$$y_lin_pred := \text{swarmfit_predict}(x_lin, \text{"linear"}, p_lin) = \begin{bmatrix} 3.2 \\ 5.1 \\ 7 \\ 8.9 \\ 10.8 \\ 12.7 \end{bmatrix}$$

Coefficient of determination R^2 for linear calibration baseline

$$R2_lin := \text{swarmfit_r2}(y_lin, y_lin_pred) = 1$$

Augmented matrix of calibration standards [X, Y]

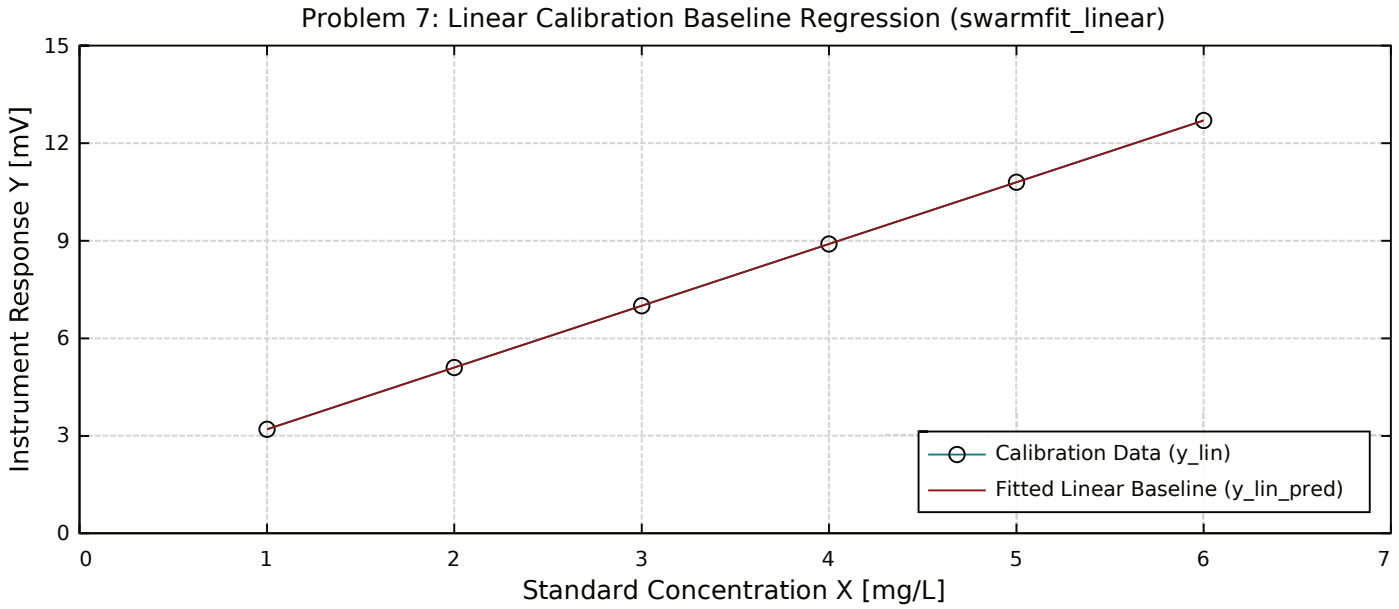
$$xy_data_lin := \text{augment}(x_lin, y_lin)$$

Augmented matrix of fitted linear baseline [X, y_lin_pred]

$$xy_fit_lin := \text{augment}(x_lin, y_lin_pred)$$

Not for commercial use

Created using a free version of SMath Studio




$$\begin{cases} xy_data_lin \\ xy_fit_lin \end{cases}$$

==========

SwarmFit Chemical & Process Engineering Verification Summary:

1. Built-in Native Model (Michaelis-Menten Kinetics, "mm"): R2_mm > 0.999 + visual fit plot

2. Built-in Exponential Relaxation (CSTR Tracer Washout, swarmfit_expdecay): R2_decay > 0.999 + visual fit plot

3. Built-in Power Law Isotherm (Freundlich Adsorption, swarmfit_power): R2_power > 0.998 + visual fit plot

4. User-Defined SMath Function (Haldane Substrate Inhibition, f_haldane): R2_haldane > 0.998 + visual fit plot + '=' cached

5. User-Defined SMath Function (Langmuir Monolayer Adsorption, f_langmuir): R2_lang > 0.998 + visual fit plot + '=' cached

6. User-Defined SMath Function (Consecutive Batch Kinetics A->B->C, f_consec): R2_consec > 0.998 + visual fit plot + '=' cached

7. Fast Analytical Linear Baseline (Spectrophotometry, swarmfit_linear): R2_lin > 0.999 + visual fit plot

8. Numerical diagnostics verified (SSE, RMSE, MaxResidual, Iterations, Improvements)

9. All variables carry descriptive engineering hover notes and tooltips

==========

Not for commercial use